Course Name: Compiler Construction
Course Code: TE7751
Faculty: Engineering
Course Credit: 3
Course Level: 4
Sub-Committee (Specialization): CSE and IT
Learning Objectives:

The Students will be able to
1. Analyze the different phases in software compilation.
2. Analyze the role of a parser; categorize parser types with its implementation approaches
3. Test syntax directed definitions on the basis of attribute types.
4. Distinguish various types of storage organization and allocation strategies in run time environment.
5. Illustrate intermediate code generation in terms of intermediate languages.
6. Criticize the design issues in code generation phase.
7. Outline code optimization activities for different types of optimizations and to study advanced compilation concepts.

Books Recommended:

Book	Author	Publisher
Compilers - Principles, Techniques and Tools	A.V. Aho, R. Shethi and J.D. Ullman	Pearson Education.
Crafting a Compiler with C	Charles Fischer, Richard LeBlanc	Pearson Publication.
Modern Compiler Design	Grune D., Bal H., Jacobs C., Langendoen K.	Wiley Publication.
Modern Compiler Implementation in Java	Andrew W. Appel	Cambridge University Press.
Compiler Construction	D.M. Dhamdare	Mc-Millan.
Introduction to Compilers and Language Design	Douglas Than	Second Edition

Course Outline:

Sr. No.	Topic	Actual Teaching Hours	Contact Hours Equivalence
1	**Introduction to Compiling** Compilers Phases of a Compiler Compiler Construction Tools Role of a Lexical Analyzer Input Buffering Specification and Recognition of Tokens finite automata implication designing a lexical analyzer generator	**8**	**8**
2	**Syntax Analysis** Role of a Parser Writing grammars for context free environments	**8**	**8**

	Top-down parsing Recursive descent and predictive parsers (LL) Bottom-Up parsing Operator precedence parsing LR, SLR and LALR parsers		
3	**Syntax Directed Translation** Syntax directed definitions Construction of syntax tree and bottom-up evaluation of S - attributed definitions L-attributed definitions Top-down translation and bottom-up evaluation of inherited attributes Analysis of syntax directed definitions	**7**	**7**
4	**Run Time Environments** Source language issues Storage organization and allocation strategies Parameter passing Symbol table organizations and generations Dynamic storage allocations	**5**	**5**
5	**Intermediate Code Generation** Intermediate languages Declarations Assignment statements and Boolean expressions Case Statements Back patching Rrocedure calls	**5**	**5**
6	**Code Generation** Issues in design of a code generator and target machine Run time storage management Basic blocks and flow graphs Next use information and simple code generator Issues of register allocation Assignment and basic blocks Code generation from DAG's and the dynamic code generation algorithm	**6**	**6**
7	**Code Optimization and Advanced Compilation Techniques** Types of optimization Peephole optimization and basic blocks Loops in flow graphs Data flow analysis and equations Code improving transformation and aliases Data flow analysis and algorithms Parallelizing compilers Optimizations and static single assignment Application of parsers Parsers and natural language processing	**6**	**6**
	Total	**45**	**45**

Pre Requisites:

None

Evaluation:

1. Continuous Assessment

i. Unit Tests

ii. Seminars

iii. Assignments

iv. Quizzes

v. Mini project

2. End Semester Examination
i. Written examination

Pedagogy:

Class Room Teaching
Practice Exercises on Regular Expressions, Automata
Seminars

Expert:

Dr. Rahul Joshi,Assistant Professor,SIT,Lavale
Dr. Shraddha Phansalkar,Assistant Professor,SIT,Lavale